DOLORES Library

Exploiting a new graphics mode on the Apple IIe

An easy to use interface to double LORES graphics for the AppleSoft programmer

Introduction

In the original Apple II LORES graphics was limited to 40 x 48 pixels in 16 colors. With the introduction of the **Apple //e enhanced** onwards a **double LORES mode** with **80 x 48 pixels in 16 colors** was implemented. However, documentation about this mode is hard to find and ways to access this mode are fairly complicated. There is no dedicated command set for AppleSoft programmers to use this mode. Instead machine language tricks are needed in order to exploit this new graphic mode on an Apple //e.

If you are a lucky owner of an Apple IIc or IIgs double LORES can be accessed easier: with some POKE-commands the double LORES mode can be activated and then standard PLOT-commands from AppleSoft can be used for drawing. However, the SCRN-function which returns the color of a pixel is reported to yield unreliable results in this mode.

In my young days as a kid I was already fascinated from this “secret” graphics mode and I always wanted to write my own tools in order to access this "*hidden*" graphics mode. However, it took me about 35 years before I could implement my ideas in assembly language.

After some first experiments using only assembly language for accessing that graphic mode I was pointed to the idea by some fellow Apple II enthusiasts to create a library that makes use of the AppleSoft **&-feature** (Ampersand) which generates an easy link from AppleSoft Basic to new machine routines.

The library first only consisted of functions for switching the mode on and off and the possibility to draw a single pixel. But soon after sharing the first results with the Apple II community new ideas emerged and the library has grown and now includes functions for drawing graphics primitives like pixels, lines, rectangles and circles, perform fast graphic page copies, support for mixed mode with four lines of 80-columns text, double buffering and last but not least a floodfill algorithm.

Double LORES – some technical tidbits

LORES screen memory is in the same memory range as the text memory and is also organized in two display pages where page 1 ranges from $0400 to $07FF and page 2 from $0800 to $0BFF. Each screen byte is divided into two nibbles which represent two pixels on the screen which are displayed as two rectangles on top of each other. The low nibble of a byte defines the color of the upper rectangle and the high nibble the color of the lower rectangle. Each rectangular pixel can be assigned one color value in the range from 0 to 15.

In this way a text screen with 40 x 24 characters is transformed into a graphics screen with 40 x 48 pixels in single LORES mode. In double LORES mode 80 x 48 pixels are available which need double the memory as single LORES mode. This is the point where we need to access the AUX (auxiliary) memory (upper 64 kB of RAM of 128 kB total system memory). This can be performed by using certain softswitches which toggle the memory banks in the Apple. While writing to AUX memory is easy while still reading the program commands from MAIN memory as a supported memory access mode, reading back the information from AUX memory is a bit tricky since it is not possible to stay in MAIN memory for program execution while just reading data back from AUX memory. In this case it is necessary to place some program code in AUX memory and transfer program code execution control to AUX memory. This is necessary e.g. for the double LORES SCRN()-function.

So, half of the double LORES screen memory is located in MAIN and the other half in AUX. However, the screen data distribution is not a simple left vs. right half or upper vs. lower half of the screen in MAIN and AUX memory. Screen data is divided in even (0, 2, 4…) and odd (1, 3, 5…) rows. Even rows are stored in MAIN memory and odd rows in AUX memory. To make it even more complicated color values (0...15) in MAIN memory must change in AUX memory when displaying the same color. The nibble needs to be rotated one position to the right (ROR) in order to keep the color appearance, the same when plotting a pixel in an uneven row, e.g. color value 3 (0011) in MAIN becomes to 9 (1001) in AUX! For speed reasons these (and other) bit shifting operations are done using a fast lookup table solution.

Entering and exiting double LORES mode is completely managed by the library. Starting from 40-columns of text first the 80-column card in slot 3 is activated. Then single LORES graphic mode is switched on and finally annunciator 3 is toggled on which gets us into the double LORES mode. Exiting double LORES mode is done by basically doing these steps in reverse order.

After implementing the PLOT-, HLIN- and VLIN-commands the need for a fast oblique line drawing mechanism was mandatory. An obvious approach was the implementation of a fast Bresenham algorithm. Since there is also a good Bresenham algorithm for fast circle drawing, circles and filled discs were also added as graphic primitives. Rectangles and boxes have been realized by using a number of HLIN- and VLIN-operations.

After implementing a fast screen fill it became obvious that a flood fill algorithm for filling irregular objects would be a nice extra feature for the library. Flood fill, also called seed fill, is an algorithm that determines the area connected to a given node in a multi-dimensional array. It is used in the "bucket" fill tool of paint programs to fill connected, similarly-colored areas with a different color. The flood-fill algorithm takes three parameters: a start node, a target color, and a replacement color. The algorithm looks for all nodes in the array that are connected to the start node by a path of the target color and changes them to the replacement color. There are many ways in which the flood-fill algorithm can be structured, but they all make use of a queue or stack data structure, explicitly or implicitly.

A very lean implementation is a stack-based recursive implementation. However, stack space is limited on the Apple II to 256 bytes which are not sufficient for larger fill areas. Hence I implemented a queue-based method which builds a search tree while stepping to the neighbors of a pixel until a fill limit condition is met and the search algorithm stops. This

approach yields rather large search trees (up to 6 kB for a large area fill) and as a side effect processes many pixel multiple times. In order to optimize this, I have implemented a secondary “visited” tree in AUX memory where all pixels are stored that have already been processed. If the evaluation of the search tree results a pixel that has been processed the algorithm skips the PLOT-procedure in order to plot the pixel in the desired new flood fill color. This approach saves around 40% of the cycles initially needed for the simple search tree approach with multiple pixel processing. Of course, this implementation is far from being optimal, but it works smoothly, fast and reliable.

Library structure

The library itself is basically a large chunk of code & tables residing from $8000 in memory and is about 5.5 kB in size (exactly 5664 bytes) and reaches unfortunately in the moment into the DOS 3.3 buffer area. This requires setting MAXFILES to a value smaller than 3 in order to prevent unwanted side effects when the library is loaded and after that disk operations are performed.

The following list gives some more information about the individual parts of the library:

- **JuMP-table for direct library function access**: At the beginning of the library is a reserved and partial free space with JMP-instruction that allow for direct access of the single functions. The vector addresses will remain fixed even when the library gets more functions or single functions will be redesigned. Hence it is possible to code your own project using the DOLORES-library and always use the next updates without the need to change your calling code. A detailed list with explanations and addresses (also with addresses of input variables) can be found on my website (see below)
- **Ampersand interface**: a large chunk of code is used by the interface code which deals with the Ampersand-routine (hook at $03F5).In order to hook the library to that vector the library needs to be BRUNned at startup. This will on the one hand set the correct vector address to point the “&” to the librarie’s Ampersand parser and on the other hand set up the AUX-memory handling code and copy parts of the executable code into AUX memory when functions are used that need to read from AUX memory (see above).
  The Ampersand-interface routine uses functions of the AppleSoft-command interpreter in order to check the input commands and data for validity and copying the input parameters to the system variables and finally calling the appropriate drawing function.
  This is on the one hand very convenient and the included range check makes it easier for AppleSoft users since they get a feedback on invalid inputs and the AppleSoft programs are very short and easier to overview. On the other hand the parsing and range checking takes some cycles. It has been reported to me that using the JuMP-table access library operations seem to be more than two times faster compared to the Ampersand-entry.
- **Drawing features**: The largest chunk of code is occupied by the drawing functions itself. The library supports the following operations (see my website for a full explanation of the functions):

    - **Mode switches**: switching between text (&TEXT), double lores graphics (&GR) and 80-column mixed mode (&M)
    - **Plotting functions**: single pixels (&PLOT), vertical lines (&VLIN), horizontal lines (&HLIN), oblique lines (&L), circles and filled disks (&C), rectangles and filled boxes (&R), setting the drawing color (&COLOR=) and reading the color of a pixel (&SCRN()).
    - **Advanced functions**: the library supports full double buffering of two double LORES graphic pages (&S and &D) which is page flipping while displaying a page and drawing on the other and flip them flickerless by evaluating the vertical blanking signal ($C019). Furthermore, the library supports a fast screen fill (&H) and a seeded flood fill algorithm (&F) as well as a fast copy routine of screen data (&SAVE) from page 1 to page 2 or 3 and vice-versa. Page 3 was allocated as an extra background buffer. When page 1 and 2 are used for page flipping animations a third screen area might become handy for the programmer as intermediate storage or to build up a new screen during the ongoing action on page 1 and 2. Using the &SAVE feature allows for fast copying the contents between the pages resulting in new possibilities for e.g. game programming.
- **Tables and variable data area**: The end of the library is occupied by table and variable data. In order to speed up the operations of the library some crucial data is stored as lookup tables. For example, the transition of colors from MAIN to AUX mem and vice-versa (as described above in order to keep the same color of a pixel a nibble needs to be RORed or ROLed). Another example would be the Y-line base addresses which are also stored as lookup table data. For some graphical operations it is necessary to flip the nibble in a byte. This can be done by bit-shifting operations or here a bit faster by using lookup-tables. The end of the code is the beginning of a small intermediate variable dump space and by now this is a part that reaches into the DOS 3.3 buffer area.

Coding examples

The following AppleSoft-Basic programs shall demonstrate the usage of the library. The first listing is a short HELLO-program that loads and activates the library as well as poking several important values with the result that the AppleSoft code and variable space does not interfere with the library itself or graphic page data storage areas. This is the way how the library should be initialized:

```
10 REM HELLO
12 HIMEM: 32768
13 PRINT CHR$(4);"MAXFILES 1"
15 POKE 104,16
16 POKE 4096,0
20 PRINT CHR$(4);"BRUN DLCOREAMP,A$8000"
25 PRINT CHR$(4);"RUN KALEIDOSCOPE"
30 END
```

The next listing shows a simple kaleidoscope program which can similarly be found on the DOS 3.3 system master disk for single LORES. This is my double LORES version:

```
10 REM DOLORES KALEIDOSCOPE
15 REM LOAD DLCOREAMP FIRST!
20 E= -16384
30 & GR
40 FOR W = 3 TO 50 : W1 = W / 73 : FOR I = 1 TO 19 : W2 = I *
W1 : I1 = 3 / (I + 3): FOR J = 0 TO 19 : K = I + J
45 IF PEEK (E) > 128 THEN 300
50 C = J * I1 + W2
60 & PLOT20+I,3+K,C:& PLOT 20+K,3+I,C:& PLOT 60- I,43-K,C:&
PLOT 60-K,4 3 - I,C
70 & PLOT 20+K,43-I,C:& PLOT 60-I,3+K,C:& PLOT 20+ I,43-K,C:&
PLOT 60-K,3 + I,C
80 NEXT J,I,W
90 GOTO 40: REM RESTART
300 & TEXT : END
```

The last example is a short demo featuring the flood fill algorithm by drawing circles and parts of circles whose centers are off-screen and afterwards filling ten randomly chosen intersection areas with a random color:

```
10 REM FILL TEST
15 REM DEMONSTRATION OF THE FLOODFILL-ALGO
20 & GR
25 FOR I = 1 TO 5
30 & COLOR= 15
40 & C INT(10+60* RND(1)),INT(6+36* RND(1)),INT(10+30* RND(1))
45 NEXT I
50 FOR I = 1 TO 10
57 XF = INT(80* RND(1))
58 YF = INT(48* RND(1))
60 & SCRN( XF,YF,C%)
61 IF C% <> 0 THEN 57
62 & COLOR= INT (14 * RND (1) + 1)
63 & F XF,YF
100 IF PEEK (-16384) > 128 THEN 300
110 NEXT I
280 FOR I = 1 TO 1000 : NEXT I
290 GOTO 20
300 & TEXT : END
```

Projects using DOLORes

There are also some projects that already use the DOLORES library as double LORES engine:

- **Magic Pixels:** a fast double LORES drawing program (https://www.crowcousins.com/magic-pixels.html)
- **Murder Manor:** a double LORES crime adventure (https://www.crowcousins.com/murder-manor.html)

Download DOLORES

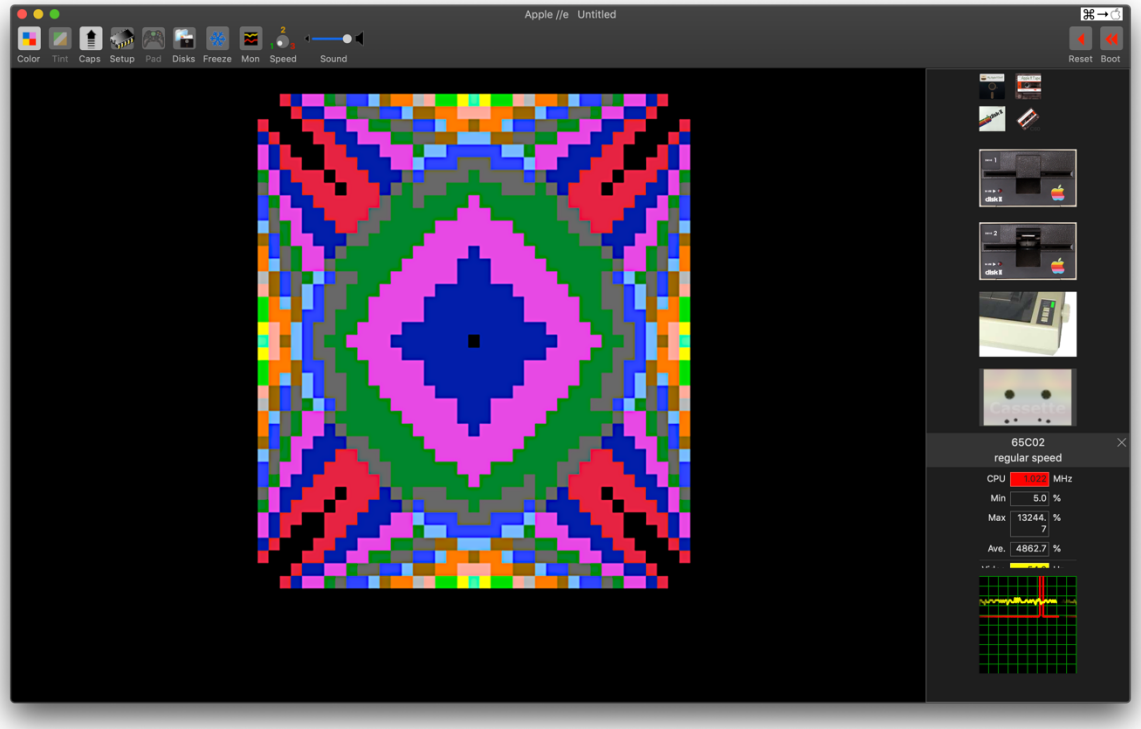
The following link takes you directly to the DOLORES project page with a full technical description and a how-to for using the library:
http://www.golombeck.eu/index.php?id=48&L=1

A DSK-image with the latest stable release of DOLORES and a bunch of demo programs can be downloaded here:
http://www.golombeck.eu/fileadmin/downloads/DLAMPCORE11.1.dsk

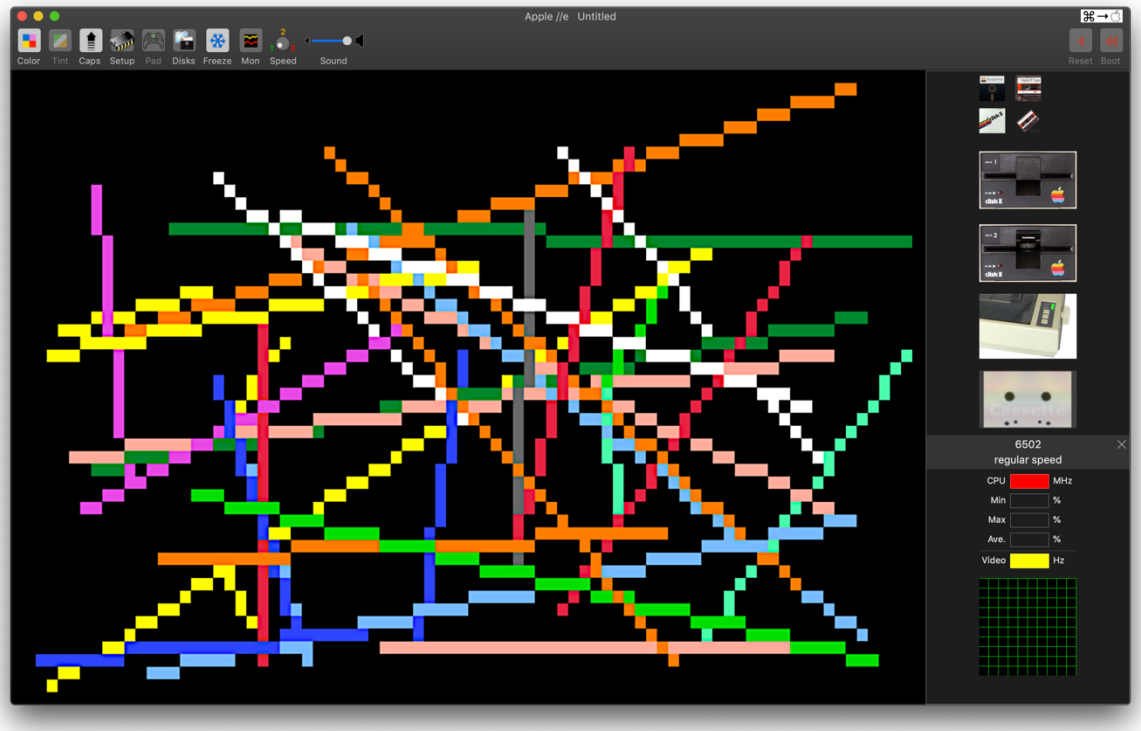
Acknowledgements

The author wishes to express his gratitude to the vivid Apple II enthusiast's scene and the valuable input that greatly influenced the development and evolution of the DOLORES library to its present state. Special thanks goes to the Roby Sherman of the CrowCousins whose feedback led to significant improvements of the library and Daniel Henderson for beta-testing and helping with bugfixing. Apple II forever!

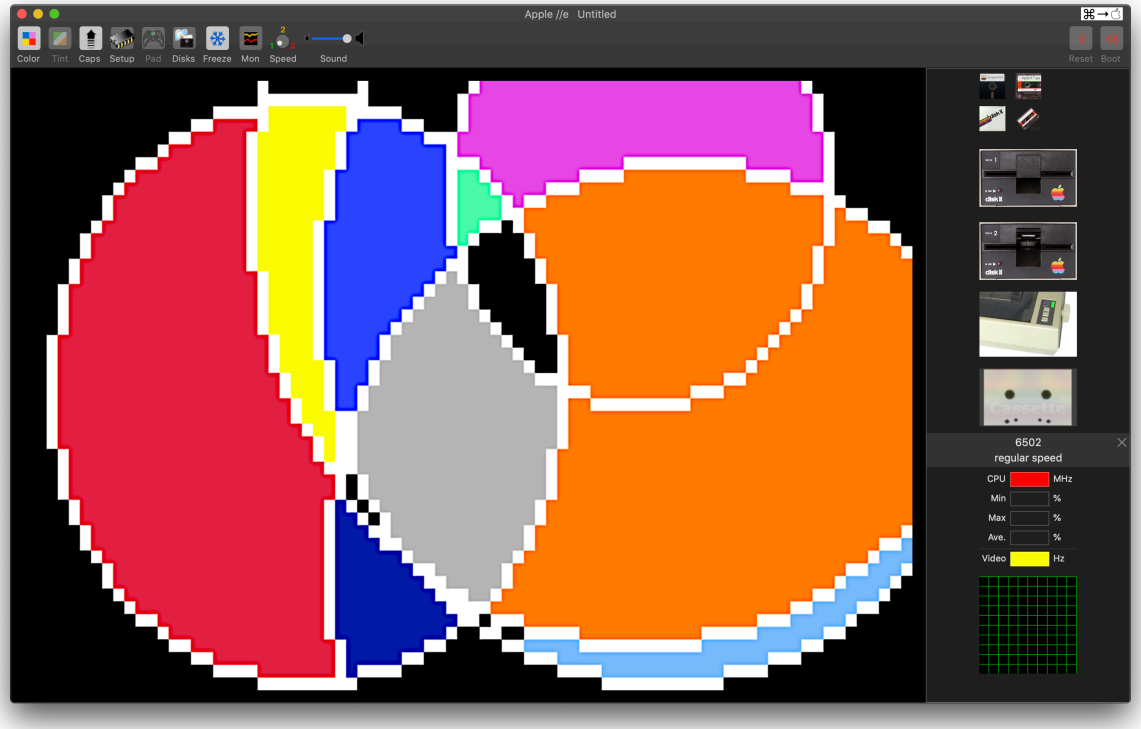
Screenshots



Kaleidoscope in double LORES



Oblique line drawing demo



Flood fill demo



Magic Pixels: double LORES drawing program powered by the DOLORES library